

Identifying Relevant Claims in Tweets

Daniel Aarnes Grønner
Universitetet i Stavanger
da.gronner@stud.uis.no

Fatema Tuz Zohora
Universitetet i Stavanger
ft.zohora@stud.uis.no

ABSTRACT

CLEF CheckThat! Lab provides different datasets targeting different tasks to identify relevant claims in tweets. Among the various tasks, we are focusing on the task of checking the worthiness of the tweets as well as detecting verifiable factual claims. Both of these tasks play a pivotal role in identifying the tweets as relevant. We choose to use Convolutional Neural Network (CNN) and long short-term memory (LSTM) for this purpose. The primary challenge was to generate better predictions with this small volume of dataset. Other challenges we faced includes finding a better way to balance the datasets and finding the best way to create the word embedding vectors that we feed into our neural networks. We could attain approximately 53.64% for checking the worthiness of the claims and 73.98% for detecting the verifiable factual claims.

1 INTRODUCTION

In the 21st century, social media's rise has transformed how we access and share information. While it provides a convenient and efficient way to communicate with a large audience, it also creates challenges in verifying the authenticity and reliability of shared information. The spread of false or misleading information can cause harm and confusion and undermine public trust in information sources.

The CLEF CheckThat! Lab seeks to address these issues through several tasks that aim to categorize the quality of the relevant claims in tweets. The tasks are designed to check the worthiness of tweets, verify factual claims, detect if tweets are harmful to society, and detect to which degree the tweets should get the attention of the policymakers, etc[4]. Overall, the CLEF CheckThat! Lab tasks are critical in promoting accuracy and reliability in online communication. Each of these tasks has its own set of advantages. For example, the spread of misinformation can be curbed by identifying and fact-checking tweets that contain false or misleading information. Additionally, by identifying verifiable factual claims, accurate information can be shared, and the public can make informed decisions based on evidence. These outcomes promote authenticity and trustworthiness in online communication, ultimately supporting a more informed and educated society.

Identifying if the tweets are authentic is crucial in ensuring that the mass people receive accurate and reliable information. To ensure this, the check-worthiness of tweets task involves predicting whether a tweet is accurate or requires fact-checking. The prediction heavily depends on participants to identify tweets containing false or misleading information that requires fact-checking. This is the preliminary task to the subsequent tasks as, if a tweet requires fact-checking, it might be benign or, to a certain extent, harmful.

As a result, the check-worthiness of tweets leads to verifying fact-checking. However, they are two completely separate tasks. Verifiable factual claims detection involves identifying whether a tweet contains a verifiable factual claim. By fact-checking claims

and highlighting inaccuracies, misinformation can be countered, and the spread of false information can be prevented. On the other hand, inaccurate or unverified claims can contribute to the spread of false information. Therefore, identifying and verifying factual claims is essential in promoting truth and trustworthiness in online communication. Moreover, identifying verifiable factual claims is vital in supporting evidence-based decision-making in various fields, from journalism to policymaking.

Therefore, in this project, we choose to focus on these critical tasks. The dataset provided by CLEF CheckThat! Lab for these tasks requires participants to analyze the content of tweets and make informed decisions based on the available information. Thus, these tasks involve supervised pattern detection. However, these datasets do not provide sufficient data for implementing deep learning algorithms, as well as, biased. Therefore, we aim to overcome these challenges and experiment with different data mining algorithms to observe which predicts the most accurate and reliable results.

2 PROBLEM STATEMENT

In this study, we aim to compare the performance of the LSTM and CNN models in predicting the binary labels to check worthiness and detect verifiable factual claims. In addition, we will evaluate the models' performance in terms of F1-score, recall, precision, and Receiver Operating Characteristic (ROC) curves.

The datasets involved in these studies are labeled and biased. Our overall experiment involves cleaning the required data, balancing the biases, and predicting classes accurately and reliably. The challenges will be training the model to extract accuracy and reliability with precision on these small-scale imbalanced data.

The results of this study will provide insights into the effectiveness of LSTM and CNN models in identifying relevant claims in tweets. This study will also contribute to developing more accurate and reliable methods for identifying false or misleading information and verifying factual claims in social media.

3 DATASET

The dataset we are using for this project is provided by the CLEF CheckThat! Lab. They have created a separate, dedicated dataset for each of the tasks they were targeting. Among them, as we are focusing on checking worthiness and detecting verifiable factual claims in the tweets, we are going to describe only these two datasets.

3.1 Check worthiness of tweets

The training set for this task consists of 2122 tweets, with 2 classes. Class 0 contains 1675 samples, while the remaining samples belong to class 1. The aim of this task is to predict whether a tweet requires fact-checking or not. The dataset for checking the worthiness of the tweets is represented below in Figure 1.

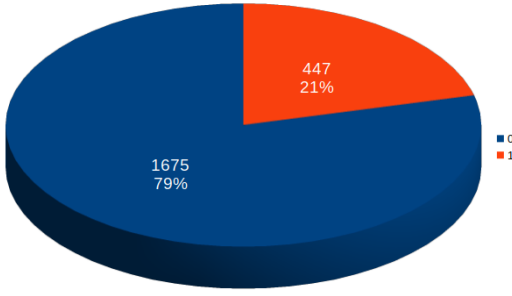


Figure 1: Dataset for checking the worthiness of the tweets

Each sample in the training set consists of the following attributes:

- **Topic:** The topic of the tweet.
- **Tweet ID:** The unique identifier for the tweet.
- **Tweet URL:** The URL for the tweet.
- **Tweet Text:** The text of the tweet.
- **Class Label:** Label on each tweet whether they are worth fact-checking.

For training the models, we only use the **Tweet Text** and **Class Label** attributes. The **Tweet Text** attribute contains the text of the tweet, which is the main feature that the models will use to make predictions. The **Class Label** attribute indicates whether a tweet is worth fact-checking or not, with "Yes" indicating that the tweet is worth fact-checking and "No" indicating that it is not. These two attributes are the most important for training the models for checking the worthiness of the tweets, as they contain the information necessary to make predictions for this task.

3.2 Verifiable factual claims detection

The task of verifiable factual claims detection performs prediction based on the labeled dataset whether they have factual claims or not. The training set for this task consists of 3325 tweets, with 2 classes. Class 1 contains 2122 samples which are approximately 64% of the total dataset. We can clearly observe an imbalance in this dataset as well. The dataset for detecting the verifiable factual claims in the tweets are represented below in Figure 2.

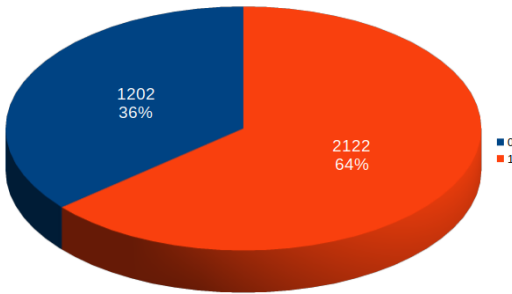


Figure 2: Dataset for detecting the verifiable factual claims in the tweets

Similar to the task of worth-checking, each sample in the training set for this task consists of the following attributes:

- **Topic:** The topic of the tweet.
- **Tweet ID:** The unique identifier for the tweet.
- **Tweet URL:** The URL for the tweet.
- **Tweet Text:** The text of the tweet.
- **Class Label:** Label on each tweet depending on detecting verifiable factual claims.

We will only focus on using the **Tweet Text** and **Class Label** attributes. As they are sufficient to make predictions for this task, these two attributes are the most important for training the models. The **Tweet Text** attribute contains the text of the tweet, which is the main feature that the models will use to make predictions. The **Class Label** attribute indicates whether a tweet has detected any verifiable facts claims with "Yes" and "No" indicating that it has not.

3.3 Pre-processing

Before selecting the model and performing the training on them, we performed data pre-processing like balancing the dataset, removing URLs, removing special characters from the tweet text attribute's values, and tokenizing the text value, etc.

3.3.1 Imbalance in the Dataset. As observed earlier, our training sets are imbalanced and biased towards a class. This imbalance can lead to biased models that perform poorly on the less represented class. Therefore, to address this issue, we can use resampling techniques such as oversampling or undersampling to create a balanced dataset. However, since the size of class 1 is relatively small compared to class 0, oversampling may lead to overfitting, and undersampling may lead to loss of information. Therefore, we will train the models on the original dataset and evaluate their performance in both classes.

3.3.2 Cleaning and Stemming Dataset. To improve the quality of the dataset, we cleaned and tokenized the tweet text to prepare the data for model training. We used the Natural Language Toolkit (NLTK) library in Python for this purpose. We performed the following cleaning steps using regular expressions:

- **Remove URLs:** We removed URLs from the tweet text using regular expressions.
- **Remove mentions and hashtags:** We removed mentions and hashtags from the tweet text using regular expressions.
- **Remove special characters:** We removed special characters from the tweet text using regular expressions.
- **Remove extra spaces:** We removed extra spaces from the tweet text using regular expressions.

We further performed the following steps to clean and stem the dataset:

- **Tokenize text:** We tokenized the tweet text by splitting it into words.
- **Remove stop words:** We removed stop words such as "and", "the", and "a" from the tokenized text.
- **Stemming:** We performed stemming on the remaining tokens using the Porter stemmer algorithm.

The preprocessing steps are important because they help to normalize the text and remove noise and irrelevant information. By removing URLs, mentions, hashtags, special characters, and stop words, we can focus on the most important words and phrases

in the tweet text. Additionally, by performing stemming, we can reduce the number of unique words in the dataset, which can help to reduce the dimensionality of the feature space and improve model performance.

4 IMPLEMENTATION

In our implementation, we used LSTM and CNNs with default parameters and with parameter tuning. We tried other models like random forests classifier, transformers, etc. However, we decided on the two mentioned algorithms for our experiments.

4.1 Model selection

LSTM (Long Short-Term Memory) is a type of recurrent neural network (RNN) that is designed to handle the vanishing gradient problem in traditional RNNs [2]. It can capture long-term dependencies in sequential data, which makes it a good choice for text classification tasks where the meaning of a sentence or document depends on the sequence of words.

CNN (Convolutional Neural Network) is a neural network commonly used in computer vision tasks. However, it has also been applied to text classification tasks and has shown promising results. CNNs can be used to learn local patterns in the input data, which can be useful for text classification tasks where important information may be present in specific text parts.

The choice between LSTM and CNN models depends on the specific task and dataset. LSTM models are better suited for tasks where the sequence of the input data is important, while CNN models are better suited for tasks where the local patterns in the input data are important. In our implementation, both models are used to explore their effectiveness in the given text classification task.

LSTM model takes in word indices as input, which is then converted to their corresponding word embeddings. The LSTM architecture in this implementation has multiple layers, which can capture increasingly complex relationships between the input sequence and the output labels.

In the CNN model, it takes in word indices as input, which is then converted to their corresponding word embeddings. The model uses multiple convolutional filters with different sizes to capture different n-gram features. The resulting features are then concatenated and passed through a fully connected layer for classification.

Overall, we choose to work with these neural network models for our project over the ensemble techniques. Firstly, the learning rate for these models is way suited for mining information, and training the model is faster than training the ensemble model. Over that, neural networks can give us better predictions when we fine-tune the models. In our implementation, we tried to implement the selected model with default parameters, as well as, we implemented it for hyper-parameter tuning. For this model, we are using the pre-trained GloVe embeddings. This provides a way to initialize text sequences with semantically meaningful representations, which can improve the model's performance.

4.2 GloVe Embeddings

GloVe stands for global vectors for word representation. GloVe is an unsupervised learning algorithm for obtaining vector representations for words [6]. Stanford developed this word representation mechanism depending on the insights to capture directly the local and global correlation of the words i.e. global corpus statistics to construct the word vectors [7]. Training is performed on aggregated global word-word co-occurrence statistics from a corpus, and the resulting representations showcase interesting linear substructures of the word vector space as shown in Figure 3 [6].

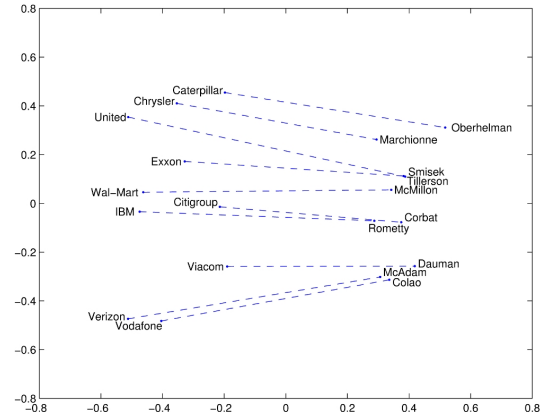


Figure 3: GloVe embeddings linear substructures

As GloVe incorporates global corpus statistics, this is better suited for analogy tasks than conventional word embeddings like word2vec [5]. In our implementation, we are trying to establish a prediction relation between the tweet text and the class label after analyzing the tweets. Therefore, we believe, GloVe is better suited for our tasks. The embeddings matrix generated can differ in size. As a result, we are padding zeros to make the inputs to the model same size.

4.3 LSTM Model

In general, the problem with Recurrent Neural Networks (RNN) is that they have a short-term memory to retain previous information with the current information in the neurons and the ability also decreases very quickly for longer sequences [?]. The LSTM models were introduced to be able to retain past information even longer as well as remove the long-term dependency. It is a special kind of recurrent neural network which is very useful for sequential data like time series data and NLP data [14].

The LSTM model controls the flow of information with three gates, namely the input gate, the output gate, and the forget gate [?]. the forget gate is responsible for selecting the important data from the current and previous layer, the input gate is responsible for weights the current input and deciding its importance, and finally, the output gate generates the output for the LSTM. There are various types of LSTM models. Figure 4 [14] depicts a single forward LSTM layer.

For our implementation, the output of the embedding layer is fed into a bi-directional LSTM layer with a specified number of hidden

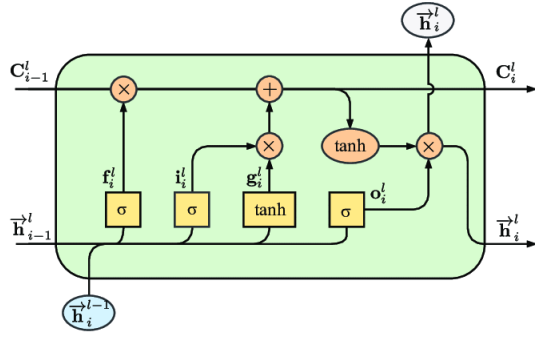


Figure 4: Single forward LSTM layer

units and layers, and the final hidden state is passed through a fully connected layer with a softmax activation function to generate the model’s output. We use the Adam optimizer with a specified learning rate and weight decay to train the model. The cross-entropy loss function is used to compute the loss during training, and we use the class weights computed earlier to balance the contribution of each class to the loss. We train the model for a specified number of epochs and monitor its performance on a held-out validation set after each epoch. If the validation loss does not improve after a specified number of epochs, the training is stopped early to prevent overfitting.

4.4 CNN Model

The second model used in the project is a convolutional neural network (CNN) implemented in PyTorch [9]. To handle huge amounts of data, we use Convolutional neural networks (CNN) as it constitutes multiple layers of artificial neural networks that currently surpass classical methods in performance, such as pattern recognition [1] [3].

The CNN takes as input a sequence of word indices and learns to classify the text based on these inputs. The architecture of the model consists of an embedding layer, followed by one or more convolutional layers, a fully connected layer, and a final output layer. The embedding layer learns a representation of each word in the input sequence as a vector of fixed length. This representation is obtained by mapping each word index to a corresponding row in a matrix of learned embeddings. The convolutional layers consist of a set of filters, each of which slides over the sequence of embeddings, producing a feature map. Each feature map represents the result of convolving the filter with the input sequence at a particular position. The filters are designed to learn various patterns and features from the input sequence, such as n-grams, and can capture local interactions between words. The max-pooling layer then selects the maximum value from each feature map to produce a feature vector. This is done to reduce the size of the feature map and capture the most important information from each feature map. The fully connected layer takes the concatenated feature vectors from all the max-pooled feature maps and applies a non-linear activation function (ReLU) to each element [13] [12]. The output of this layer is then fed into the final output layer, which produces the probability of the input belonging to a certain class. Figure 5 represents how different layers of convolutional neural network are connected.

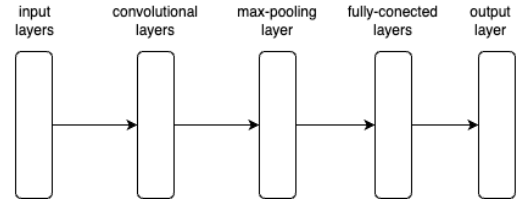


Figure 5: Single forward LSTM layer

The model is trained using cross-entropy loss and the Adam optimizer for our implementation [10] [8]. Dropout is applied to prevent overfitting, and ReLU activation functions are used to introduce non-linearity to the model [11] [13]. Overall, the CNN model is a powerful tool for text classification and has been shown to perform well on a variety of natural language processing tasks.

5 EXPERIMENTS AND RESULTS

Hyperparameter optimization is performed for two distinct types of neural network models, namely Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN), applied to two separate tasks (check the worthiness of the tweets and detect the verifiable factual claims in tweets). The Optuna library facilitates the optimization process, with the objective function aiming to maximize the evaluation metric, either the F1 score or accuracy, depending on the task. The models are tuned to optimize the F1 score to check the worthiness of the tweets (a) and accuracy in detecting the verifiable factual claims in tweets (b), as recommended by CLEF CheckThat! Lab.

The hyperparameter tuning process commences with importing the required libraries. We saved the trial results in a CSV file, ensuring the organization and accessibility of trial data. The core of the optimization process is encapsulated in the objective function, which receives a trial, model type, and for which task it will be implemented as input arguments. The function suggests common hyperparameters shared by both model types are suggested, including learning rate, dropout rate, number of epochs, and patience for early stopping.

Model-specific hyperparameters are suggested based on the model type. For the LSTM model, the hidden dimension, while for the CNN model, the number of filters, filter sizes, and hidden dimension is proposed. The argument dictionary is updated with the suggested hyperparameters, and the execution starts using these arguments. This returns an evaluation metric, which is either the F1 score or accuracy, contingent upon the task.

In a nutshell, the following steps are undertaken to predict the class and evaluating their accuracy:

- An iteration is conducted through the model types (LSTM and CNN) and tasks (a and b).
- An Optuna study is created, aiming to maximize the evaluation metric.
- The study is optimized by running the objective function for each trial, totaling 200 trials.
- The best trial results are printed, encompassing the optimal hyperparameters for each model type and subtask combination.

This structured approach to hyperparameter optimization provides a systematic means of identifying the most suitable model architecture and hyperparameter configuration for the given tasks, ultimately enhancing model performance, which was the goal of our experimentation. The results of our experiments are summed up in the tables below.

This table depicts how experimentation details of best and worst-case scenarios we found when we were experimenting with Long Short-Term Memory for checking the worthiness of the tweets.

Hidden	Lr	Dropout	Patience	F1
98	0.0005	0.0730	6	0.5183
52	0.000003	0.3247	5	0

Table 1: LMST for checking the worthiness of the tweets

This table depicts how experimentation details of best and worst-case scenarios we found when we were experimenting with convolutional neural networks for checking the worthiness of the tweets.

	Max	Min
Filters	130	16
Ngrams	[2]	[6,5,6]
Hidden	29	71
Lr	0.0003	0.000002
Dropout	0.036	0.230
Patience	5	10
F1	0.5364	0

Table 2: CNN for checking the worthiness of the tweets

This table depicts how experimentation details of best and worst-case scenarios we found when we were experimenting with Long Short-Term Memory for detecting the verifiable factual claims in tweets.

Hidden	Lr	Dropout	Patience	Accuracy
74	0.0009	0.2079	6	0.7398
75	0.00006	0.4450	8	0.3699

Table 3: LMST for detecting the verifiable factual claims

This table depicts how experimentation details of best and worst-case scenarios we found when we were experimenting with convolutional neural networks for detecting the verifiable factual claims in tweets.

From our experimentation, we found out that CNN works slightly better for checking the worthiness of the tweets, while LSTM works slightly better for detecting verifiable factual claims.

6 DISCUSSION AND FUTURE WORKS

We choose to work with these neural networks because we believe these models can perform better than ensemble techniques when hyperparameter tuning is considered. For that reason, we implemented both these models with parameter tuning. We represented

	Max	Min
Filters	72	64
Ngrams	[7, 3, 5, 2, 2, 3]	[7,3,3]
Hidden	54	53
Lr	0.0006	0.001
Dropout	0.1809	0.2556
Patience	7	7
Accuracy	0.7311	0.6125

Table 4: CNN for detecting the verifiable factual claims

the best and the worst result we found for each model after parameter tuning mentioning different parameters that were involved. We are using n-grams to find complex patterns and relationships in the text which depend on the position of the word in that sentence. For that reason, we choose to use word indices instead of TF-IDF because it preserves word order and context, whereas TF-IDF only considers a word as important depending on the repetition of that word in that text.

For our future work, we can try out different architectures of our selected algorithms. Additionally, we tried to incorporate transformers for this project, but we could not make it work in time.

7 CONCLUSION

In conclusion, we would consider CNN for predicting the worthiness of the tweets. The best result we generated using this model for this task was 0.5364, while we would consider LSTM for detecting the verifiable factual claims in tweets, as the most accurate result we observed was generated by this model (0.7398). The difference in the generated results by these two models is not much. We believe the reason is the volume of data, as it was a very small dataset for the neural networks. With more data, the results may change as well as the performance of these models.

REFERENCES

- [1] Saad Albawi, Tareq Abed Mohammed, and Saad Al-Zawi. 2017. Understanding of a convolutional neural network. In *2017 international conference on engineering and technology (ICET)*. Ieee, 1–6.
- [2] Avishek Anand. [n.d.]. Recurrent Neural Networks. https://stavanger.instructure.com/courses/11759/files/1420853?module_item_id=263073
- [3] Vijay Choubey. [n.d.]. Text classification using CNN. <https://medium.com/voice-tech-podcast/text-classification-using-cnn-9ade8155dfb9>
- [4] CLEF2022. [n.d.]. Task 1: Identifying Relevant Claims in Tweets. <https://sites.google.com/view/clef2022-checkthat/tasks/task-1-identifying-relevant-claims-in-tweets>
- [5] Thushan Ganegedara. 2019. *Intuitive Guide to Understanding GloVe Embeddings*. <https://towardsdatascience.com/light-on-math-ml-intuitive-guide-to-understanding-glove-embeddings-b13b4f19c010>
- [6] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. <https://nlp.stanford.edu/projects/glove/>
- [7] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.
- [8] pytorch. [n.d.]. ADAM. <https://pytorch.org/docs/stable/generated/torch.optim.Adam.html>
- [9] Pytorch. [n.d.]. Conv1d. <https://pytorch.org/docs/stable/generated/torch.nn.Conv1d.html>
- [10] pytorch. [n.d.]. CrossEntropyLoss. <https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>
- [11] pytorch. [n.d.]. Dropout. <https://pytorch.org/docs/stable/generated/torch.nn.Dropout.html>
- [12] pytorch. [n.d.]. maxpool1d. https://pytorch.org/docs/stable/generated/torch.nn.functional.max_pool1d.html

- [13] pytorch. [n.d.]. ReLU. <https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html>
- [14] Yugesh Verma. [n.d.]. Hands-On Guide to Bi-LSTM With Attention. <https://analyticsindiamag.com/hands-on-guide-to-bi-lstm-with-attention/>

A CONTRIBUTIONS

For implementation, Fatema Tuz Zohora was responsible for the CNN and Daniel Grønner was responsible for the LSTM. However,

Daniel Grønner was more responsible for the hyperparameter tuning, and Fatema Tuz Zohora was more responsible for writing the report.

B PROGRAM LISTINGS

Github: <https://github.com/Danielgronner/dat550>